

元语言的严谨定义与罗素悖论的避免

摘要

本文以严格的元语言视角，为一阶逻辑的语法与语义提供完整的元理论刻画。我们首先对元语言本身做出非形式化但精确的定义，明确其包含的基本逻辑、自然数理论、朴素集合论的安全片段以及字符串与语义概念。在此基础上，深入分析元语言为何不会导出罗素悖论：从实践层面、原理层面和可严格化层面三重保障加以论证。最后给出元语言的简明特征画像，凸显其作为逻辑基础工具的温和性与安全性。

目录

1	引言	2
2	元语言的严谨定义	2
2.1	元语言的基本逻辑	2
2.2	元语言的自然数	2
2.3	元语言的集合概念	3
2.3.1	朴素集合论概述	3
2.3.2	元语言中使用的集合操作（安全片段）	4
2.4	字符串与语法	5
2.5	语义概念	6
3	元语言如何避免罗素悖论	7
3.1	第一层：实际使用中从不构造“所有集合的集合”	7
3.2	第二层：分离公理模式的限制使用	7
3.3	第三层：可严格化到无悖论的元理论	8
4	总结	8

1 引言

在为—阶逻辑（first-order logic）构造语法和语义的元理论时，我们必须使用一种位于对象语言之外的语言——称为元语言（meta-language）。这种元语言不同于完全形式化的对象语言（如 ZFC 公理集合论或—阶谓词演算），它是一种非形式化但精确描述的数学实践语言。正是凭借元语言，我们才能谈论符号、公式、证明、结构、满足等概念，从而赋予形式系统以意义。

然而，若元语言本身包含悖论（如罗素悖论），则整个逻辑大厦将崩塌于基础。因此，一个核心问题是：元语言为何免于罗素悖论？

本文严格聚焦于元语言本身。全文不使用类比或比喻，而是从元语言的构成成分出发，逐一界定其能力边界，并证明在此边界内罗素悖论无法导出。

2 元语言的严谨定义

元语言是日常精确化的数学语言，由以下五个模块构成：基本逻辑、自然数理论、受限的集合操作、字符串与语法、语义概念。下面逐一严格界定。

2.1 元语言的基本逻辑

元语言的最低限度逻辑能力包括命题连接词与量词。

定义 2.1 (命题连接词). 元语言包含以下命题连接词，其意义遵循经典二值逻辑：

且 ($\wedge$), 或 ($\vee$), 若...则... ($\rightarrow$), 当且仅当 ($\leftrightarrow$), 并非 ($\neg$).

定义 2.2 (量词). 元语言包含全称量词与存在量词：

对所有 x ($\forall x$), 存在 x ($\exists x$).

这些量词的变域涵盖元语言中所讨论的任意对象：自然数、符号、字符串、公式、集合等。量词的使用不受类型限制——这是元语言区别于类型论的关键特征。

定义 2.3 (等词). 元语言包含等词 “=”，表示对象的同一性 (*identity*)，满足：

(i) 自反性: $\forall x (x = x)$;

(ii) 对称性: $\forall x \forall y (x = y \rightarrow y = x)$;

(iii) 传递性: $\forall x \forall y \forall z ((x = y \wedge y = z) \rightarrow x = z)$;

(iv) 可替换性: 若 $x = y$ ，则在任何元语言表达式中可用 y 替换 x 而保持真值不变。

2.2 元语言的自然数

元语言预设一套关于自然数的非形式理论，其强度大致相当于原始递归算术（PRA）。

定义 2.4 (自然数的基本设定). 元语言接受：

(i) 一个起始对象 0 (零)；

(ii) 一个后继运算 $'$ ：对每个自然数 n ， n' 是其唯一后继；

(iii) 归纳原理：若性质 P 满足 $P(0)$ 且 $\forall n (P(n) \rightarrow P(n'))$ ，则 $\forall n P(n)$ 。

注记 2.1 (实无穷与潜无穷). 自然数全体 $\mathbb{N}$ 在标准数学元语言中被视为一个已完成的无穷集合（实无穷立场）。若持构造主义立场，则 $\mathbb{N}$ 仅作为一个生成过程（潜无穷）存在。本文采用标准的实无穷立场，这是经典模型论通行的选择。

定义 2.5 (原始递归定义). 元语言允许对自然数进行原始递归定义。例如加法 $+$ 与乘法 $\times$ 可由以下递归方程定义：

$$\begin{aligned} n + 0 &= n, & n + m' &= (n + m)', \\ n \times 0 &= 0, & n \times m' &= (n \times m) + n. \end{aligned}$$

由此可进一步定义阶乘、指数、前驱等一切原始递归函数。

注记 2.2. 归纳原理与原始递归定义赋予元语言足够的表达力以编码语法对象（如哥德尔编码），这为第三层保障（可严格化到 PRA）提供了基础。

2.3 元语言的集合概念

元语言使用朴素集合论的受限片段，而非完整的朴素集合论。在进入元语言的具体集合操作前，有必要先澄清朴素集合论本身的含义及其问题所在。

2.3.1 朴素集合论概述

定义 2.6 (朴素集合论). 朴素集合论 (*naive set theory*) 是由格奥尔格·康托尔 (*Georg Cantor*) 在 19 世纪创立的原始集合论。其核心原理是概括公理 (*Comprehension Principle*):

对任意可表达的性质（谓词） $P(x)$ ，存在一个集合 S 恰好由所有满足 P 的对象构成，记作

$$S = \{x \mid P(x)\}.$$

换言之，任意性质决定一个集合。这一原理极其直观：我们只需用语言描述一个汇集的条件，该汇集便被承认为一个数学对象——集合。

例 2.1. 在朴素集合论中，以下均被认可为集合：

$$\{x \mid x \text{ 是偶数}\}, \quad \{x \mid x \text{ 是集合}\}, \quad \{x \mid x \notin x\}.$$

第三个例子直接通向悖论。

定义 2.7 (罗素悖论). 令 $R = \{x \mid x \notin x\}$ 。由概括公理， R 是一个集合。现在考察 $R \in R$ 是否成立：

- 若 $R \in R$ ，则根据 R 的定义， R 的元素必须满足 $x \notin x$ ，故 $R \notin R$ ，矛盾。
- 若 $R \notin R$ ，则 R 满足条件 $x \notin x$ ，故 $R \in R$ ，矛盾。

因此，朴素集合论在经典逻辑下是不一致的。

注记 2.3 (历史注记). 罗素于 1901 年发现此悖论，随后独立地由策梅洛 (*Zermelo*) 发现。这一发现直接推动了公理化集合论 (ZFC、NBG 等) 的诞生，其核心修复策略是用严格的公理控制集合的存在，而非允许任意性质决定集合。

概括公理: $\forall P \exists S (x \in S \leftrightarrow P(x))$

$$\begin{aligned}
 &\Downarrow \\
 &\text{取 } P(x) := x \notin x \\
 &\Downarrow \\
 &R = \{x \mid x \notin x\} \\
 &\Downarrow \\
 &R \in R \leftrightarrow R \notin R \\
 &\Downarrow \\
 &\text{矛盾!}
 \end{aligned}$$

2.3.2 元语言中使用的集合操作（安全片段）

元语言并不拥抱完整的朴素集合论，而是仅使用其一个严格受限的安全片段。该片段的全部操作罗列如下。

定义 2.8 (原子对象). 元语言中预设一些基本的原子对象 (*urelements*)，它们本身不是集合，但可以作为集合的元素。典型的原子对象包括：

符号: “ v ”, “ 0 ”, “ c ”, “ f ”, “ $\forall$ ”, “ $\exists$ ”, “ $\rightarrow$ ”, “(”, “)”, ...

定义 2.9 (有穷集合的枚举构造). 可以从已有对象通过枚举形成有穷集合：

$$\{a_1, a_2, \dots, a_k\}.$$

该集合的元素恰为所列出的对象。

定义 2.10 (归纳生成的集合). 元语言接受如下形式的归纳定义：一个集合 S 是满足下述闭包条件的最小集合：

- (i) S 包含某些指定的初始元素（基础条款）；
- (ii) S 在若干生成规则下封闭（归纳条款）：若 $e_1, \dots, e_n \in S$ ，则由它们按规则构成的复合对象也属于 S ；
- (iii) 没有其他对象属于 S （极小性条款）。

下面的交换图刻画了归纳集合的逐层构造过程（以克林闭包为例）：

$$S_0 \hookrightarrow S_1 \hookrightarrow S_2 \hookrightarrow \dots \hookrightarrow S = \bigcup_{n \in \mathbb{N}} S_n$$

其中 S_0 为基础元素集， S_{n+1} 为对 S_n 施加所有生成规则所得结果的并集， S 的极小性保证没有多余元素。

定义 2.11 (子集与分离). 对于已存在的集合 X ，允许根据元语言中可表达的确切性质 P 分离出子集：

$$\{x \in X \mid P(x)\}.$$

关键限制：分离必须指定一个来源域 X ，绝不允许不指定域而直接写 $\{x \mid P(x)\}$ 。这是元语言区别于朴素集合论概括公理的核心安全机制。

定义 2.12 (有序对与关系). 有序对 (a, b) 可以视为原始概念, 也可通过库拉托夫斯基对 (Kuratowski pair) 在集合论内部定义为

$$(a, b) := \{\{a\}, \{a, b\}\}.$$

函数 $f: X \rightarrow Y$ 定义为满足以下条件的二元关系 $f \subseteq X \times Y$:

$$\forall x \in X \exists! y \in Y ((x, y) \in f).$$

进而可构造笛卡尔积、函数空间 Y^X 等。

定义 2.13 (有限序列与克林闭包). 对于已存在的集合 X , X 中元素的有限序列的全体称为 X 的克林闭包 (Kleene closure), 记作 X^* , 其归纳定义如下:

- (i) 空序列 $\varepsilon \in X^*$;
- (ii) 若 $s \in X^*$ 且 $x \in X$, 则 $s \frown x$ (将 x 添加到 s 末尾) 属于 X^* ;
- (iii) 仅由 (i)(ii) 生成的序列属于 X^* 。

注记 2.4 (全集假设的否定). 元语言中从不断言存在一个包含一切集合或一切对象的全集 (universal set)。集合总是从某些已给定的原子或集合出发, 通过上述受限制的操作逐步构建而成。这从根本上切断了罗素悖论的生成路径。

2.4 字符串与语法

在拥有了集合操作能力之后, 我们可以严格地定义一阶逻辑的语法对象。

定义 2.14 (字母表). 语法构造的出发点是符号 (字母表) Σ , 它是元语言中预设的原子对象的有限或可数集合。典型的一阶逻辑字母表包含:

- 逻辑符号: $\forall, \exists, \rightarrow, \neg, =, (,)$;
- 变元符号: $v_0, v_1, v_2, \dots$ (可数无穷多个互不相同的原子);
- (可选的) 常元符号、函数符号、谓词符号: 由具体语言决定。

定义 2.15 (字符串). Σ 中元素的有限序列称为 Σ 上的字符串。全体字符串的集合为克林闭包 Σ^* , 由上一小节的归纳定义保证其存在。

定义 2.16 (项与公式 (归纳定义)). Term (项集) 和 Form (公式集) 作为 Σ^* 的特定子集, 通过如下分离条件 (即满足归纳闭包条件的最小集合) 加以定义:

项集 Term 是满足以下条件的最小集合:

- (i) 每个变元 $v_i \in \text{Term}$;
- (ii) 每个常元符号 $c \in \text{Term}$;
- (iii) 若 f 为 n 元函数符号且 $t_1, \dots, t_n \in \text{Term}$, 则 $f(t_1, \dots, t_n) \in \text{Term}$ 。

公式集 Form 是满足以下条件的最小集合:

- (i) 若 $t_1, t_2 \in \text{Term}$, 则 $(t_1 = t_2) \in \text{Form}$ (原子公式);
- (ii) 若 R 为 n 元谓词符号且 $t_1, \dots, t_n \in \text{Term}$, 则 $R(t_1, \dots, t_n) \in \text{Form}$;

- (iii) 若 $\varphi \in \text{Form}$, 则 $(\neg\varphi) \in \text{Form}$;
- (iv) 若 $\varphi, \psi \in \text{Form}$, 则 $(\varphi \rightarrow \psi) \in \text{Form}$;
- (v) 若 $\varphi \in \text{Form}$ 且 v_i 为变元, 则 $(\forall v_i \varphi) \in \text{Form}$ 。

Term 和 Form 作为 Σ^* 的子集, 是通过在 Σ^* 上应用分离公理获得的, 因而是元语言中的合法集合。

下面的交换图展示了从符号出发到公式的构造层次:

$$\Sigma \xrightarrow{\text{克林闭包}} \Sigma^* \xrightarrow{\text{归纳分离}} \text{Term} \xrightarrow{\text{归纳分离}} \text{Form}$$

注记 2.5. 在 Term 和 Form 的归纳定义中, “最小集合” 条件等价于: 一个字符串属于 Term (或 Form) 当且仅当它可以通过有限次应用生成规则从基础元素得到。这一性质在元语言中可通过自然数的归纳原理严格证明 (对构造步数做归纳)。

2.5 语义概念

语法定义了符号系统的合法表达式, 语义则赋予这些表达式以意义——将它们映射到某个数学结构中。

定义 2.17 (结构). 一个一阶逻辑的结构 (*structure*, 亦称解释或模型) $\mathfrak{A}$ 由以下成分组成:

- 一个非空集合 A , 称为论域 (*universe/domain*);
- 对每个常元符号 c , 指定一个元素 $c^{\mathfrak{A}} \in A$;
- 对每个 n 元函数符号 f , 指定一个函数 $f^{\mathfrak{A}}: A^n \rightarrow A$;
- 对每个 n 元谓词符号 R , 指定一个关系 $R^{\mathfrak{A}} \subseteq A^n$ 。

集合 A 及上述函数、关系均为元语言中的合法对象。

定义 2.18 (赋值). 赋值 (*assignment*) 是从变元集合到论域的元语言函数:

$$s: \{v_0, v_1, v_2, \dots\} \rightarrow A.$$

定义 2.19 (项的指称). 对赋值 s 和项 t , 其指称 $t^{\mathfrak{A}}[s]$ 通过项复杂度上的归纳定义:

$$\begin{aligned} v_i^{\mathfrak{A}}[s] &= s(v_i), \\ c^{\mathfrak{A}}[s] &= c^{\mathfrak{A}}, \\ f(t_1, \dots, t_n)^{\mathfrak{A}}[s] &= f^{\mathfrak{A}}(t_1^{\mathfrak{A}}[s], \dots, t_n^{\mathfrak{A}}[s]). \end{aligned}$$

定义 2.20 (满足关系 $\models$). 对结构 $\mathfrak{A}$ 、赋值 s 和公式 φ , 满足关系 $\mathfrak{A} \models \varphi[s]$ 通过公式复杂度上的归纳定义如下:

$$\begin{aligned} \mathfrak{A} \models (t_1 = t_2)[s] &\iff t_1^{\mathfrak{A}}[s] = t_2^{\mathfrak{A}}[s], \\ \mathfrak{A} \models R(t_1, \dots, t_n)[s] &\iff (t_1^{\mathfrak{A}}[s], \dots, t_n^{\mathfrak{A}}[s]) \in R^{\mathfrak{A}}, \\ \mathfrak{A} \models (\neg\varphi)[s] &\iff \text{并非 } \mathfrak{A} \models \varphi[s], \\ \mathfrak{A} \models (\varphi \rightarrow \psi)[s] &\iff \text{若 } \mathfrak{A} \models \varphi[s] \text{ 则 } \mathfrak{A} \models \psi[s], \\ \mathfrak{A} \models (\forall v_i \varphi)[s] &\iff \text{对所有 } a \in A, \mathfrak{A} \models \varphi[s(v_i \mid a)]. \end{aligned}$$

其中 $s(v_i \mid a)$ 表示将赋值 s 在变元 v_i 处的值修改为 a 而其余不变。定义中出现的元语言量词 “对所有 $a \in A$ ” 是合法的, 因为 A 是一个已存在的确定集合。

下面的交换图总结了语法、语义及其关系：

$$\begin{array}{ccccccc}
 \Sigma & \longrightarrow & \Sigma^* & \longrightarrow & \text{Form} & \longrightarrow & \varphi \\
 & & & & \uparrow \text{满足关系} \models & \downarrow \text{语义赋值 } s & \\
 & & & & \mathfrak{A} = (A, \dots) & &
 \end{array}$$

至此，一阶逻辑的完整语法与语义均在元语言内被定义完毕。下面我们分析这一元语言构造为何免于逻辑悖论。

3 元语言如何避免罗素悖论

罗素悖论的根源在于无限制的概括公理模式：允许对任意性质 $P(x)$ 形成集合 $\{x \mid P(x)\}$ ，尤其当 $P(x)$ 取 $x \notin x$ 时导致矛盾。

元语言不会导出此悖论，原因可从三个层面理解：实践层面、原理层面和可严格化层面。

3.1 第一层：实际使用中从不构造“所有集合的集合”

在为—阶逻辑提供元理论定义时，所需构造的集合极端具体且范围明确：

- 符号集 Σ ——由我们显式列出的有限或可数无穷集；
- 克林闭包 Σ^* ——有限字符串的集合；
- 项集 Term 和公式集 Form——作为 Σ^* 的特定子集；
- 对给定的结构 $\mathfrak{A}$ ，只考虑论域 A 及由 A 通过分离、配对、并集、幂集等受控操作得到的集合。

换言之，元语言操作的始终是一个良基的、有明确生成基础的集合宇宙片段——通常是由原子符号出发、通过上述受限操作迭代构建的集合层次。没有任何步骤需要、也没有任何定义诱使我们构造一个“包含一切集合的集合”或“所有序数的集合”这样的全域对象。

注记 3.1. 这与 ZFC 集合论中通过累积层级 V_α 构造集合宇宙的精神一致：集合总是从底层逐步生成，不存在自指的全域。

3.2 第二层：分离公理模式的限制使用

元语言中看似朴素的集合操作，本质上全部是受限制的概括（restricted comprehension）：

元语言从不单独写 $\{x \mid P(x)\}$ 而不指定其来源域。

每次形成集合，都限定在已事先存在的集合 X 内做分离，即形式为

$$\{x \in X \mid P(x)\}.$$

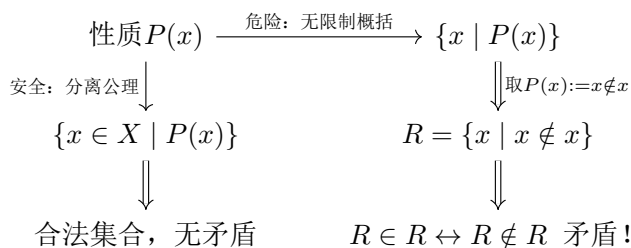
这直接阻断了罗素悖论的推理。因为罗素悖论的精髓在于允许构造“包含所有不包含自身的集合的那个集合” $R = \{x \mid x \notin x\}$ ——这个构造恰是缺少来源域限制的无限制概括。

在元语言中，公式集 Form 的定义本质上就是在 Σ^* 上应用分离：

$$\text{Form} = \{s \in \Sigma^* \mid s \text{ 满足公式的归纳闭包条件}\}.$$

类似地，所有集合构造都有明确的父集（parent set）作为语境域。

下面的交换图对比了两种概括方式的本质差异：



3.3 第三层：可严格化到无悖论的元理论

如果希望对元语言本身也赋予完全形式化的基础（从而彻底消除对朴素集合的依赖），可以将上述元语言严格化到以下两类无悖论的系统中：

- (1) **原始递归算术 (PRA)**：在 PRA 中，根本没有“集合”这一对象类型。取而代之的是自然数和定义在自然数上的谓词，通过哥德尔编码将符号、字符串、公式等语法对象编码为自然数。归纳定义和归纳证明均可归约为对自然数的原始递归定义和归纳。
- (2) **有限类型论 (finite type theory)**：使用严格分层的类型结构——个体属于类型 0，个体集合属于类型 1，个体集合的集合属于类型 2，以此类推。此时 $x \in y$ 仅在 y 的类型比 x 高一级时才是合式公式，从而 $x \in x$ 根本不被语法所允许——罗素悖论在语法层面即被消解。

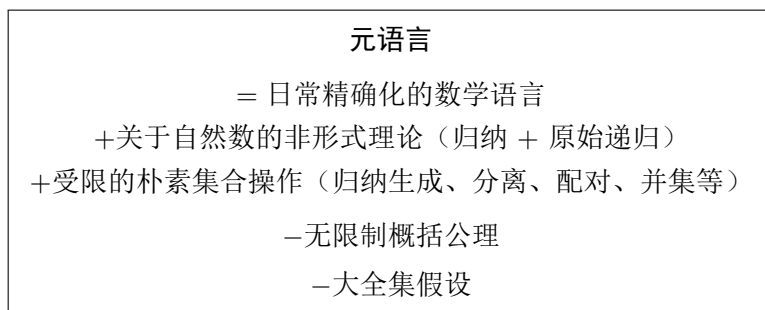
无论是 PRA 还是有限类型论，其一致性都已在极弱的元理论中被证明（PRA 的一致性证明甚至可在命题逻辑加无量词归纳的系统中完成）。因此，我们可以确信：

标准数学实践中使用的元语言，可以在不引入悖论的前提下被完全形式化。

元语言所调用的集合论操作——归纳生成集合、在已有集合上分离、配对、并集等——都可以在 ZFC 去掉无穷公理（甚至更弱的系统中）完成，而该弱系统的相对一致性已被确立。

4 总结

经过上述三个层面的分析，我们可以给出元语言的精确特征画像：

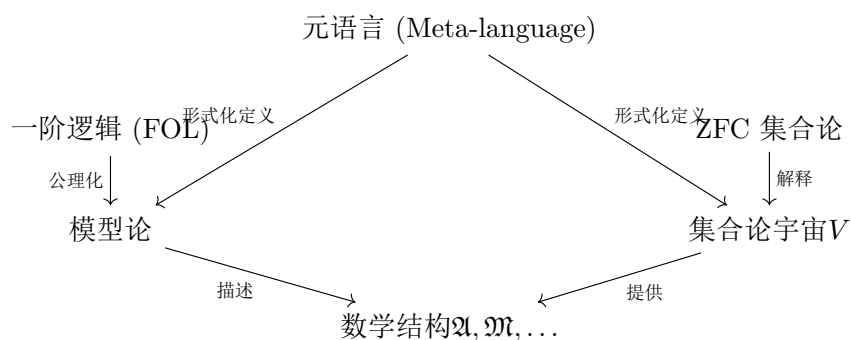


其安全性的核心不在于某种形式化的“防护措施”，而在于其实践用法的天然边界：元语言的任务永远是构造具体的、有底的语法和语义对象（符号、字符串、项、公式、结构、满足关系），而不涉及自指的全域构造。

注记 4.1 (对象语言与元语言的层次分离). 一旦我们需要研究“集合”这个概念本身（如集合的公理化、基数的性质、连续统假设等），我们便跳入对象语言（如 ZFC 公理化集合论），在其中用严格的

公理控制集合的存在。此时，原来的元语言退居幕后，成为研究该形式系统的工具语言。这种严格的层次分离——元语言研究形式系统，形式系统研究数学对象——天然地避免了语义悖论。

下面的交换图刻画了这种层次分离结构：



该图表明：元语言同时形式化定义了一阶逻辑的语法/语义和 ZFC 集合论的公理系统。一阶逻辑作为逻辑框架被 ZFC 所使用（ZFC 是一阶理论），而 ZFC 所描述的集合论宇宙 V 又为我们研究数学结构提供了丰富的论域。元语言始终处于这一层次结构的顶端，它的任务是指定规则，而非成为自身规则的应用对象。

总之，元语言免于罗素悖论不是侥幸，而是其设计本性的必然结果：它的能力恰好足以支撑语法和语义的归纳定义，却不足以产生自指悖论——这是一种精确校准后的恰到好处的强与恰到好处的弱。